# Dynamic Memory

Matteo Fasiolo

# Previously

- A pointer stores an address.
- Dereferencing a pointer accesses its target.
- Array elements occupy contiguous memory.
- If `p` points to the first element, `p[i]` or `*(p + i)` access later elements.

So far we used **fixed-length arrays** whose size is known when writing the program (at compile-time).

Today we focus on **dynamic arrays** whose length is known only while the program runs (at run-time).

# Today

By the end of the lecture, you should be able to:

1. distinguish local storage from dynamically allocated storage;
2. allocate a runtime-sized array using `malloc`;
3. initialise and process the allocation;
4. release it exactly once using `free`;
5. recognise common dynamic-memory errors.

# Motivation: a data-size problem

Suppose a program records customer ratings.

The number of ratings is entered by the user:

```
int count;
printf("Enter the number of ratings: ");
scanf("%d", &count);
// I won't ask you about scanf() !
```

When writing and compiling the program, we do not know whether `count` will be 1, 100 or 10,000.

So far we used a size known when writing the program:

```
double ratings[5];
```

Now the required length is known only at runtime.

Some C implementations do support variable-length arrays:

```
int count;
scanf("%d", &count);
double ratings[count];
```

This might compile and run.

# So... why go further?

A variable length array looks like the whole answer.

But there are two further questions:

1. can such an array outlive the function that created it?
2. what happens when `count` is large?

We answer these in turn: dynamic allocation will answer both.

# Recall: the call stack

The call stack adds one frame for each active function call and removes it when the call returns.




Parameters and other local variables exist only while their function call is active.

Two commonly used terms:

- **scope** says where a name may be used
- **lifetime** says how long a variable or allocated array exists.

# Local variables are managed automatically

```
void example(void) {
    int x = 10;
    double values[5];
}
```

Conceptually:

- the variables are created for the function call;
- no explicit memory allocation is needed;
- they stop existing when the call ends.

In the call-stack model, these variables are stored in the function's frame.

# A local array cannot outlive its function

Suppose we want a function that builds an array and returns it:

```
double *make_ratings(int n) {
    double local[n];
    //... fill local ...
    return local;
}
```

`local` exists only while this function call is active.

When the call returns, the caller is handed an address that no longer points to a valid array.

So, this will produce an invalid pointer

```
double *ratings = make_ratings(100);
```

The compiler might warn: `function returns address of local variable`.

So answer to:

1. can such an array outlive the function that created it?

is **NO**.

What about:

2. what happens when `count` is large?

You might get a **stack overflow** error, because the stack is limited in size.

Your computer might have 1 MB to 8 MB of stack memory.

Instead, dynamic memory can be GBytes, depending on your RAM.

# Dynamic storage

Dynamic allocation lets a program explicitly request storage while it runs.

The allocated array:

- is not removed because the allocating function returns;
- remains allocated until the program explicitly releases it;
- is typically placed in a memory region called the **heap**.

# malloc

The main function for dynamic allocation is `malloc`.

To use it, you must include the corresponding library:

```
#include <stdlib.h>
```

`malloc` allocates a contiguous block of dynamic storage:

```
double *ratings = malloc(number_of_bytes);
```

It returns:

- a pointer to the allocated block if successful;
- `NULL` if the request cannot be satisfied (useful for checking).

# How many bytes should we allocate?

For `count` elements:

```
double *ratings =
    malloc(count * sizeof(*ratings));
```

`sizeof(*ratings)` is the size of one pointed-to element.

Alternatively, we can write

```
double *ratings = malloc(count * sizeof(double));
```

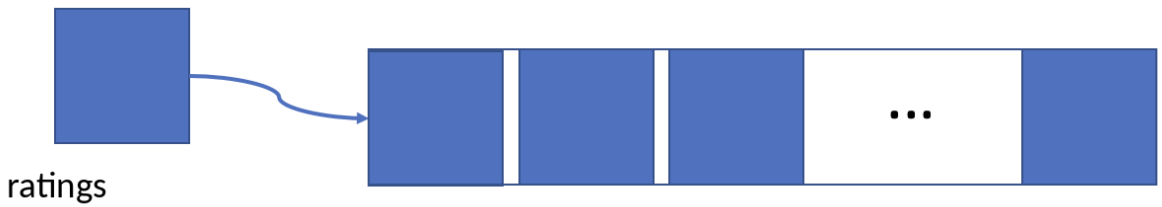
But this requires more work if we change array type:

```
int *ratings = malloc(count * sizeof(int));
```

What has this code

```
double *ratings = malloc(count * sizeof(double));
```

created?




- `ratings` stores the starting address returned by `malloc`.
- The dynamic block has space for `count` consecutive elements.
- `ratings[i]` accesses element `i` as for fixed-length arrays.

Note: `malloc` does not initialise the elements.

Do not read them before assigning values:

```
for (int i = 0; i < count; i = i + 1) {
    ratings[i] = 2.5 + 0.5 * i;
}
```

Keep in mind:

- The pointer is valid: it is initialised by `malloc`.
- The pointed-to array elements still need initialisation.

# `free`

Once you are done, release an allocation using `free`:

```
free(ratings);
```

Rules:

- pass a pointer returned by `malloc`;
- release the allocation **at most once**;
- do not access its elements afterwards.

`free(ratings)` ends the lifetime of the allocated array.

The local variable `ratings` still exists, but does not point to a valid allocation.

It is an **invalid pointer**.

# Live demonstration 1

## Switch to VS Code

Open `dynamic_ratings.c`

The complete sequence for using dynamic arrays is:

```
determine size
     -> malloc
     -> initialise (if needed)
     -> use
     -> free exactly once
```

Each allocation needs a clearly identifiable final `free`.

# What if we do not free?

If we reach the end of `main`, heap memory is released anyway.

The problem is when we allocate repeatedly with `malloc`, and do not `free`.

Example:

```
int *p;
while (1) {
    p = malloc(1000 * sizeof(*p));
    // do something with p
}
```

each iteration allocates a new block of memory.

This is a **memory leak**.

Leaks are especially harmful in long-running programs or code that repeatedly allocates memory.

# Use after free() and double free

```
free(ratings);
printf("%.1f\n", ratings[0]);  // invalid
```

The allocation's lifetime has ended.

Accessing it has undefined behaviour.

Another error is:

```
free(ratings);
free(ratings);  // invalid: memory already released
```

Calling `free` twice for one allocation has undefined behaviour.

# Three ways to obtain an array

| Question | Fixed local | VLA* local | Dynamic |
|---|---|---|---|
| Example | `double x[5]` | `double x[n]` | `double *x = malloc(...)` |
| Length | fixed by the declaration | chosen at runtime | chosen at runtime |
| Obtain memory | automatically | automatically | use `malloc` |
| Release memory | when the function returns | when the function returns | use `free` |
| Outlives its function? | no | no | yes |
| Maximum size | MBytes | MBytes | GBytes |

$*$ VLA = variable-length array.

# Check your understanding

1. Why is `malloc(10)` the wrong size for 10 doubles?
2. May an element be read immediately after `malloc`?
3. What operation actually releases the allocation?
4. What is wrong with reading `ratings[0]` after `free(ratings)`?
5. What causes a memory leak?

# Conclusion

1. Dynamic allocation provides storage whose size can be selected at runtime.
2. Pointers to the allocated memory can be returned from (or passed to) functions.
3. Allocate, initialise (if needed) and then use the elements.
4. Every allocation must be released exactly once.
5. A memory leak, use after free and double free are errors.